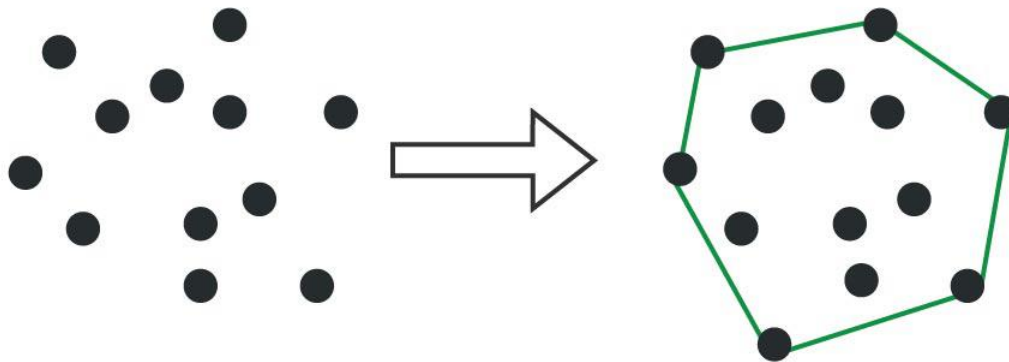


Convex Hull using Divide and Conquer Algorithm

Last Updated : 23 Jul, 2025



In computational geometry, a **convex hull** is the smallest convex polygon that contains a given set of points. It is a fundamental concept with applications in various fields such as computer graphics, robotics, and image processing.



Convex Hull



Importance of Convex Hull:

Convex hulls are important in computational geometry for several reasons:

- **Collision detection:** Convex hulls can be used to efficiently detect collisions between objects in 2D or 3D space.
- **Image processing:** Convex hulls can be used to extract meaningful shapes from images, such as the outline of an object.
- **Data visualization:** Convex hulls can be used to visualize the distribution of data points in a scatter plot.
- **Input** is an array of points specified by their x and y coordinates. The **output** is the convex hull of this set of points.

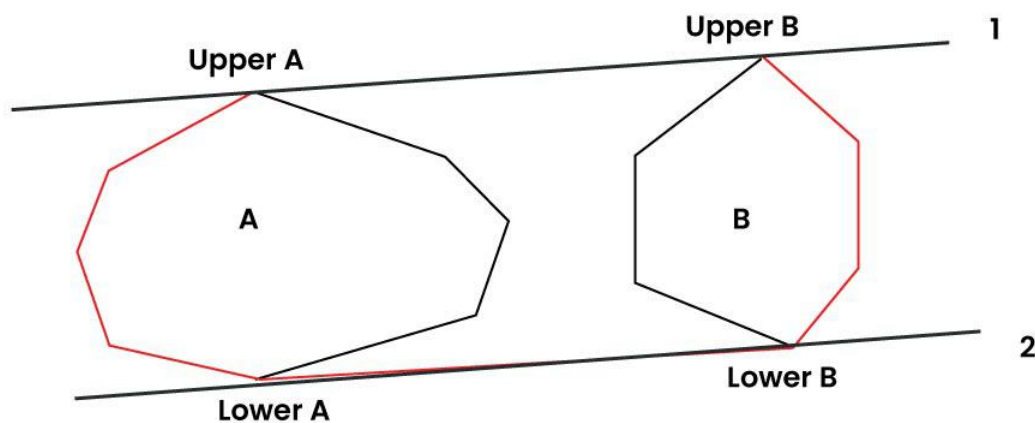
Convex Hull using Divide and Conquer Algorithm:

Pre-requisite: [Tangents between two convex polygons](#)

Algorithm: Given the set of points for which we have to find the convex hull.

Suppose we know the convex hull of the left half points and the right half points,

then the problem now is to merge these two convex hulls and determine the convex hull for the complete set. This can be done by finding the upper and lower tangent to the right and left convex hulls. This is illustrated here [Tangents between two convex polygons](#) Let the left convex hull be a and the right convex hull be b. Then the lower and upper tangents are named as 1 and 2 respectively, as shown in the figure. Then the red outline shows the final convex hull.



Final Convex Hull



Now the problem remains, how to find the convex hull for the left and right half. Now recursion comes into the picture, we divide the set of points until the number of points in the set is very small, say 5, and we can find the convex hull for these points by the brute algorithm. The merging of these halves would result in the convex hull for the complete set of points.

Note: We have used the brute algorithm to find the convex hull for a small number of points and it has a time complexity of $O(n^3)$. But some people suggest the following, the convex hull for 3 or fewer points is the complete set of points. This is correct but the problem comes when we try to merge a left convex hull of 2 points and right convex hull of 3 points, then the program gets trapped in an infinite loop in some special cases. So, to get rid of this problem I directly found the convex hull for 5 or fewer points by $O(n^3)$ algorithm, which is somewhat greater but does not affect the overall complexity of the algorithm.

Below are the implementation:

C++

Java

Python

C#

JavaScript

```

// A divide and conquer program to find convex
// hull of a given set of points.
#include<bits/stdc++.h>
using namespace std;

// stores the centre of polygon (It is made
// global because it is used in compare function)
pair<int, int> mid;

// determines the quadrant of a point
// (used in compare())
int quad(pair<int, int> p)
{
    if (p.first >= 0 && p.second >= 0)
        return 1;
    if (p.first <= 0 && p.second >= 0)
        return 2;
    if (p.first <= 0 && p.second <= 0)
        return 3;
    return 4;
}

// Checks whether the line is crossing the polygon
int orientation(pair<int, int> a, pair<int, int> b,
                pair<int, int> c)
{
    int res = (b.second-a.second)*(c.first-b.first) -
              (c.second-b.second)*(b.first-a.first);

    if (res == 0)
        return 0;
    if (res > 0)
        return 1;
    return -1;
}

// compare function for sorting
bool compare(pair<int, int> p1, pair<int, int> q1)
{
    pair<int, int> p = make_pair(p1.first - mid.first,
                                p1.second - mid.second);
    pair<int, int> q = make_pair(q1.first - mid.first,
                                q1.second - mid.second);

```

```

        q1.second - mid.second);

    int one = quad(p);
    int two = quad(q);

    if (one != two)
        return (one < two);
    return (p.second*q.first < q.second*p.first);
}

// Finds upper tangent of two polygons 'a' and 'b'
// represented as two vectors.
vector<pair<int, int>> merger(vector<pair<int, int> > a,
                             vector<pair<int, int> > b)
{
    // n1 -> number of points in polygon a
    // n2 -> number of points in polygon b
    int n1 = a.size(), n2 = b.size();

    int ia = 0, ib = 0;
    for (int i=1; i<n1; i++)
        if (a[i].first > a[ia].first)
            ia = i;

    // ib -> leftmost point of b
    for (int i=1; i<n2; i++)
        if (b[i].first < b[ib].first)
            ib=i;

    // finding the upper tangent
    int inda = ia, indb = ib;
    bool done = 0;
    while (!done)
    {
        done = 1;
        while (orientation(b[indb], a[inda], a[(inda+1)%n1]) >=0)
            inda = (inda + 1) % n1;

        while (orientation(a[inda], b[indb], b[(n2+indb-1)%n2])
<=0)
        {
            indb = (n2+indb-1)%n2;

```

```

        done = 0;
    }
}

int uppera = inda, upperb = indb;
inda = ia, indb=ib;
done = 0;
int g = 0;
while (!done)//finding the lower tangent
{
    done = 1;
    while (orientation(a[inda], b[indb], b[(indb+1)%n2])>=0)
        indb=(indb+1)%n2;

    while (orientation(b[indb], a[inda], a[(n1+inda-1)%n1])
<=0)
    {
        inda=(n1+inda-1)%n1;
        done=0;
    }
}

int lowera = inda, lowerb = indb;
vector<pair<int, int>> ret;

//ret contains the convex hull after merging the two convex
hulls
//with the points sorted in anti-clockwise order
int ind = uppera;
ret.push_back(a[uppera]);
while (ind != lowera)
{
    ind = (ind+1)%n1;
    ret.push_back(a[ind]);
}

ind = lowerb;
ret.push_back(b[lowerb]);
while (ind != upperb)
{
    ind = (ind+1)%n2;
    ret.push_back(b[ind]);
}

```

```

    }
    return ret;
}

// Brute force algorithm to find convex hull for a set
// of less than 6 points
vector<pair<int, int>> bruteHull(vector<pair<int, int>> a)
{
    // Take any pair of points from the set and check
    // whether it is the edge of the convex hull or not.
    // if all the remaining points are on the same side
    // of the line then the line is the edge of convex
    // hull otherwise not
    set<pair<int, int> >s;

    for (int i=0; i<a.size(); i++)
    {
        for (int j=i+1; j<a.size(); j++)
        {
            int x1 = a[i].first, x2 = a[j].first;
            int y1 = a[i].second, y2 = a[j].second;

            int a1 = y1-y2;
            int b1 = x2-x1;
            int c1 = x1*y2-y1*x2;
            int pos = 0, neg = 0;
            for (int k=0; k<a.size(); k++)
            {
                if (a1*a[k].first+b1*a[k].second+c1 <= 0)
                    neg++;
                if (a1*a[k].first+b1*a[k].second+c1 >= 0)
                    pos++;
            }
            if (pos == a.size() || neg == a.size())
            {
                s.insert(a[i]);
                s.insert(a[j]);
            }
        }
    }
}

```

```

vector<pair<int, int>>ret;
for (auto e:s)
    ret.push_back(e);

// Sorting the points in the anti-clockwise order
mid = {0, 0};
int n = ret.size();
for (int i=0; i<n; i++)
{
    mid.first += ret[i].first;
    mid.second += ret[i].second;
    ret[i].first *= n;
    ret[i].second *= n;
}
sort(ret.begin(), ret.end(), compare);
for (int i=0; i<n; i++)
    ret[i] = make_pair(ret[i].first/n, ret[i].second/n);

return ret;
}

// Returns the convex hull for the given set of points
vector<pair<int, int>> divide(vector<pair<int, int>> a)
{
    // If the number of points is less than 6 then the
    // function uses the brute algorithm to find the
    // convex hull
    if (a.size() <= 5)
        return bruteHull(a);

    // left contains the left half points
    // right contains the right half points
    vector<pair<int, int>>left, right;
    for (int i=0; i<a.size()/2; i++)
        left.push_back(a[i]);
    for (int i=a.size()/2; i<a.size(); i++)
        right.push_back(a[i]);

    // convex hull for the left and right sets
    vector<pair<int, int>>left_hull = divide(left);
    vector<pair<int, int>>right_hull = divide(right);

```

```

    // merging the convex hulls
    return merger(left_hull, right_hull);
}

// Driver code
int main()
{
    vector<pair<int, int> > a;
    a.push_back(make_pair(0, 0));
    a.push_back(make_pair(1, -4));
    a.push_back(make_pair(-1, -5));
    a.push_back(make_pair(-5, -3));
    a.push_back(make_pair(-3, -1));
    a.push_back(make_pair(-1, -3));
    a.push_back(make_pair(-2, -2));
    a.push_back(make_pair(-1, -1));
    a.push_back(make_pair(-2, -1));
    a.push_back(make_pair(-1, 1));

    int n = a.size();

    // sorting the set of points according
    // to the x-coordinate
    sort(a.begin(), a.end());
    vector<pair<int, int> >ans = divide(a);

    cout << "convex hull:\n";
    for (auto e:ans)
        cout << e.first << " "
                << e.second << endl;

    return 0;
}

```

Output

```

convex hull:
-5 -3
-1 -5
1 -4

```


0 0
-1 1

Time Complexity: The merging of the left and the right convex hulls take $O(n)$ time and as we are dividing the points into two equal parts, so the time complexity of the above algorithm is $O(n * \log n)$.

Auxiliary Space: $O(n)$

Related Articles :

- [Convex Hull | Set 1 \(Jarvis's Algorithm or Wrapping\)](#)
- [Convex Hull | Set 2 \(Graham Scan\)](#)
- [Quickhull Algorithm for Convex Hull](#)

 Comment

 **kartik**

 18 

Article Tags :

Divide and Conquer

Geometric

DSA

Convex Hull

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information